

Relatorio tecnico-cientifico

Deteccao de reviews falsas/sinteticas em portugues brasileiro usando CNN 1D em PyTorch

Este relatorio apresenta um experimento supervisionado de classificacao textual binaria para distinguir reviews genuinas de reviews falsas/sinteticas em portugues brasileiro. O modelo principal e uma rede neural convolucional unidimensional implementada em PyTorch, comparada com dois baselines classicos: TF-IDF + Regressao Logistica e TF-IDF + SVM Linear.

A unidade de analise e a review individual. O experimento nao afirma deteccao completa de astroturfing, pois o dataset nao contem usuario, data, rating, produto especifico, relacoes entre contas ou informacoes temporais.

1. Introducao

Reviews online influenciam a percepcao de consumidores e podem afetar decisoes de compra em plataformas de e-commerce. Nesse contexto, opinion spam, fake reviews e astroturfing sao conceitos importantes para estudar manipulacao de opiniao. Este trabalho aborda um recorte mais restrito: a classificacao de reviews individuais como genuinas ou falsas/sinteticas.

O objetivo tecnico e avaliar se uma CNN 1D, treinada diretamente sobre texto em portugues brasileiro, consegue distinguir as classes operacionais do Fake Reviews PT-BR Dataset. A classe falsa deve ser entendida como sintetica, gerada por GPT-2 no contexto do dataset, e nao como fraude humana comprovada.

2. Dataset

O experimento usa o Fake Reviews PT-BR Dataset, derivado da base publica Brazilian E-Commerce Public Dataset by Olist.

Fontes:

- Fake Reviews PT-BR Dataset: <https://github.com/cristianomg10/fake-reviews-ptbr-dataset>
- Olist: <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>

Colunas reais detectadas e registradas em **outputs/tables/column_mapping.json**:

Campo canonico	Coluna real
texto	review_comment_message
categoria	product_category_name
rotulo	label

Rotulos usados:

Rotulo	Interpretacao
0	review falsa/sintetica

Rotulo	Interpretacao
1	review genuina

3. Analise exploratoria

A EDA foi gerada automaticamente em **outputs/tables/** e **outputs/figures/**.

O resumo principal esta em **outputs/tables/eda_summary.json**.

Principais numeros observados apos a limpeza minima:

- Total de amostras: 29.988.
- Valores ausentes nas colunas canonicas: 0.
- Linhas duplicadas: 683.
- Textos duplicados: 1.537.
- Tamanho medio das reviews: 82,95 caracteres.
- Mediana de caracteres: 92.
- Tamanho medio em tokens: 13,98.
- Mediana de tokens: 15.
- Tamanho maximo observado: 272 caracteres e 45 tokens.

Distribuicao por classe, conforme **outputs/tables/class_distribution.csv**:

Classe	Quantidade
falsa/sintetica (0)	15.000
genuina (1)	14.988

A distribuicao por categoria esta em **outputs/tables/category_distribution.csv**.

A maioria das categorias manteve aproximadamente 2.000 exemplos; pequenas variacoes aparecem apos a limpeza, por exemplo **furniture_decor** com 1.997 e **electronics** com 1.998.

Figuras geradas:

- **outputs/figures/class_distribution.png**
- **outputs/figures/category_distribution.png**
- **outputs/figures/review_char_length_hist.png**
- **outputs/figures/review_token_length_hist.png**
- **outputs/figures/review_token_length_by_class.png**

4. Trabalhos relacionados

Borges et al. (2025) apresentam o benchmark associado ao Fake Reviews PT-BR Dataset, fornecendo uma base publica para classificacao de reviews falsas em portugues brasileiro. Kim (2014) mostrou que CNNs podem ser eficazes para classificacao de sentencas, explorando filtros convolucionais sobre sequencias de embeddings. Souza, Nogueira e Lotufo (2020) introduziram o BERTimbau, uma familia de modelos BERT para portugues brasileiro que pode servir como extensao futura ou comparacao posterior. Devlin et al. (2019) consolidaram o uso de Transformers pre-treinados em PLN, e Goodfellow, Bengio e Courville (2016)

fornece a base teórica geral de aprendizado profundo.

Neste trabalho, BERTimbau não é usado como modelo principal. O foco da avaliação é a implementação e análise de uma CNN 1D treinada no experimento.

5. Metodo

5.1 Pre-processamento

O pipeline realiza limpeza mínima para preservar sinais textuais:

- converte o texto para string;
- normaliza quebras de linha, tabulações e espaços consecutivos;
- remove reviews vazias;
- mantém pontuação e vocabulário original, sem limpeza agressiva.

Os dados foram divididos com estratificação em treino, validação e teste na proporção 70/15/15, usando **random_state=42**.

5.2 Baselines classicos

Foram treinados dois modelos clássicos:

1. TF-IDF + Regressão Logística.
2. TF-IDF + SVM Linear.

A AUC-ROC foi calculada tomando a classe **0** como classe de interesse. Na regressão logística, o score usado foi a probabilidade da classe **0**. No SVM linear, o score foi ajustado para representar a classe falsa/sintética.

5.3 CNN 1D em PyTorch

Arquitetura do modelo principal:

```
Embedding(vocab_size, embedding_dim, padding_idx=0)<br>-> Conv1d(in_channels=embedding_dim, out_channels=filters, kernel_size=kernel_size)<br>-> ReLU<br>-> GlobalMaxPool1d<br>-> Dropout<br>-> Linear(filters, 1)
```

A saída da rede é um logit. Para avaliação:

- **prob_label_1 = sigmoid(logits);**
- **prob_label_0 = 1 - prob_label_1;**
- **pred_label = 1** se **prob_label_1 >= 0.5**, senão **0**.

Treinamento:

- framework: PyTorch;
- loss: **BCEWithLogitsLoss**;
- otimizador: Adam;
- learning rate: 0.001;
- batch size: 64;
- limite: 30 épocas;
- early stopping por **val_loss**, **patience=4**;
- melhor modelo salvo em **outputs/models/cnn1d.pt**.

O historico real de treino esta em **outputs/tables/cnn_training_history.csv**.

O menor **val_loss** ocorreu na epoca 7, e o treinamento parou na epoca 11 apos nao haver melhora suficiente dentro da paciencia configurada.

6. Configuracao experimental

A configuracao experimental foi definida para manter reproducibilidade e comparacao direta entre os modelos.

Item	Configuracao
Linguagem	Python
Modelo neural	CNN 1D em PyTorch
Baselines	TF-IDF + Regressao Logistica; TF-IDF + SVM Linear
Divisao dos dados	treino/validacao/teste em 70/15/15
Estratificacao	por rotulo binario
Semente	random_state=42
Classe de interesse	0 , falsa/sintetica
Loss da CNN	BCEWithLogitsLoss
Otimizador da CNN	Adam
Learning rate	0.001
Batch size	64
Early stopping	monitoramento de val_loss , patience=4
Saida da CNN	logit convertido com sigmoid para probabilidades

As metricas foram calculadas no conjunto de teste. Para precisao, recall e F1, a classe **0** foi tratada como classe positiva operacional. Para AUC-ROC, o score usado foi a probabilidade ou pontuacao da classe falsa/sintetica.

7. Resultados

A tabela abaixo reproduz os resultados reais de **outputs/tables/model_comparison.csv**.

Modelo	Accuracy	Precision fake label 0	Recall fake label 0	F1 fake label 0	F1 macro	AUC-ROC fake label 0
TF-IDF + Regressao Logistica	0.885086	0.866385	0.910667	0.887974	0.885009	0.949770
TF-IDF + SVM Linear	0.897533	0.882428	0.917333	0.899542	0.897492	0.962379
CNN 1D PyTorch	0.949544	0.950557	0.948444	0.949499	0.949544	0.987807

Figuras de resultado:

- **outputs/figures/cnn_loss_curve.png**
- **outputs/figures/cnn_accuracy_curve.png**
- **outputs/figures/confusion_matrix_cnn1d.png**
- **outputs/figures/confusion_matrix_linear_svm.png**
- **outputs/figures/confusion_matrix_logistic_regression.png**

8. Analise critica

A CNN 1D PyTorch teve o melhor desempenho geral. Ela alcançou acuracia de 0,949544, F1 macro de 0,949544 e AUC-ROC de 0,987807 para a classe falsa/sintetica. Em termos percentuais, isso corresponde a aproximadamente 94,95% de F1 macro e 98,78% de AUC-ROC para a classe **0**.

Entre os modelos classicos, o SVM linear superou a regressao logistica. O SVM obteve acuracia de 0,897533 contra 0,885086 da regressao logistica, alem de F1 macro de 0,897492 contra 0,885009. Isso sugere que uma margem linear sobre representacoes TF-IDF foi mais eficaz do que a regressao logistica neste recorte.

Comparada ao SVM linear, a CNN teve ganho de aproximadamente 5,2 pontos percentuais de acuracia. A alta AUC da CNN sugere boa separacao entre reviews sinteticas e genuinas no dataset, especialmente quando a classe falsa/sintetica (**0**) e tratada como classe de interesse.

Esse resultado, entretanto, deve ser interpretado com cautela. O modelo aprende a distinguir textos genuinos de textos sinteticos gerados no processo de construcao do dataset. Isso nao equivale a detectar fraude humana real nem a identificar astroturfing completo. Para esse escopo mais amplo, seriam necessarios metadados de usuario, tempo, produto, rating, relacoes entre contas e sinais de coordenacao.

9. Analise qualitativa de erros

A analise qualitativa esta em **outputs/tables/cnn_error_analysis.csv**. O arquivo lista falsos positivos e falsos negativos da CNN:

- falsos positivos: reviews genuinas classificadas como falsas/sinteticas;
- falsos negativos: reviews falsas/sinteticas classificadas como genuinas.

Os exemplos devem ser avaliados qualitativamente no relatorio final da disciplina ou do TCC. A leitura desses casos ajuda a investigar situacoes em que reviews curtas, genericas, muito padronizadas, negativas ou com poucos detalhes concretos podem ficar mais proximas da fronteira de decisao.

Essa etapa tambem e importante para discutir ameacas a validade. Como a classe falsa foi gerada por GPT-2, o modelo pode estar aprendendo padroes do gerador ou do procedimento de construcao da base, e nao necessariamente caracteristicas universais de fraude humana.

10. Ameacas a validade e limitacoes

Principais limitacoes:

- A classe falsa e sintetica/operacional, nao fraude humana comprovada.
- A unidade de analise e apenas a review individual.
- Nao ha dados de usuario, data, rating, produto especifico ou rede de contas.
- Duplicatas existem no dataset e foram documentadas na EDA.

- O bom desempenho pode refletir diferenças entre textos da Olist e textos gerados por GPT-2, não necessariamente padrões gerais de fake reviews reais.
- BERTimbau não foi treinado neste experimento; ele permanece como comparação futura.

11. Conclusão

O experimento implementou um pipeline completo e reproduzível para classificação de reviews falsas/sintéticas em português brasileiro. A CNN 1D em PyTorch foi o modelo principal e superou os dois baselines clássicos avaliados. O SVM linear foi o melhor baseline entre os modelos TF-IDF, enquanto a CNN apresentou o melhor resultado geral, com aproximadamente 94,95% de F1 macro e 98,78% de AUC-ROC para a classe falsa/sintética.

Para o TCC, este experimento pode ser aproveitado de três formas principais:

1. Como modelo neural intermediário entre abordagens TF-IDF e uma comparação futura com BERTimbau.
2. Como pipeline experimental reutilizável para carregamento, EDA, pré-processamento, treino, avaliação, geração de figuras e análise de erros.
3. Como base para discutir ameaças à validade, especialmente a diferença entre detectar texto sintético em um dataset controlado e detectar fake reviews ou astroturfing em ambientes reais.

Trabalhos futuros podem incluir comparação com BERTimbau, avaliação em datasets externos, estudo de robustez, análise mais detalhada de erros e incorporação de metadados quando disponíveis. BERTimbau deve ser tratado como extensão ou comparação futura, não como substituto do modelo principal desta avaliação.

Referências

BORGES, Eduardo C. R. et al. Benchmarking Machine Learning Algorithms in Fake Reviews Detection in Brazilian Portuguese. *Revista Brasileira de Computação Aplicada*, 2025.

BORGES, Eduardo C. R. Fake Reviews PT-BR Dataset, 2025. Disponível em: <https://github.com/cristianomg10/fake-reviews-ptbr-dataset>.

OLIST; SIONEK, Andre. Brazilian E-Commerce Public Dataset by Olist, 2018. Disponível em: <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>.

KIM, Yoon. Convolutional Neural Networks for Sentence Classification. *EMNLP*, 2014.

DEVLIN, Jacob et al. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *NAACL-HLT*, 2019.

SOUZA, Fabio; NOGUEIRA, Rodrigo; LOTUFO, Roberto. BERTimbau: Pretrained BERT Models for Brazilian Portuguese. *BRACIS*, 2020.

GOODFELLOW, Ian; BENGIO, Yoshua; COURVILLE, Aaron. *Deep Learning*. MIT Press,

2016.